

Codificação Para Front-end

12

2026

SENAI - CRTI



Posicionamento e Layout Tradicional

Continuando o projeto de e-commerce, aprenderemos a organizar visualmente os elementos da página com técnicas clássicas de posicionamento CSS — base para um layout profissional e funcional.

SENAI · DESENVOLVIMENTO FRONT-END

Onde Estamos

Retomada: O Que Já Fizemos

Aulas Anteriores

Nas aulas anteriores, construímos a estrutura HTML do e-commerce e aplicamos os primeiros estilos CSS: cores, tipografia, espaçamentos e box model.

- Estrutura HTML semântica (header, nav, main, footer)
- CSS básico: cores, fontes e margens
- Box model: margin, padding, border
- Seletores e especificidade

O Problema Atual

O site funciona, mas os elementos ainda não estão bem organizados visualmente. Menus empilhados, cards sem alinhamento e header que some ao rolar a página são problemas comuns nessa etapa.



A estrutura existe, mas o **posicionamento visual** precisa ser melhorado. É exatamente isso que resolveremos hoje!

Problemas Comuns de Layout

Antes de aprender as soluções, é importante reconhecer os problemas mais frequentes em projetos iniciantes.

Elementos Empilhados

Itens de menu aparecem um abaixo do outro quando deveriam estar lado a lado na horizontal.

Header Sem Fixação

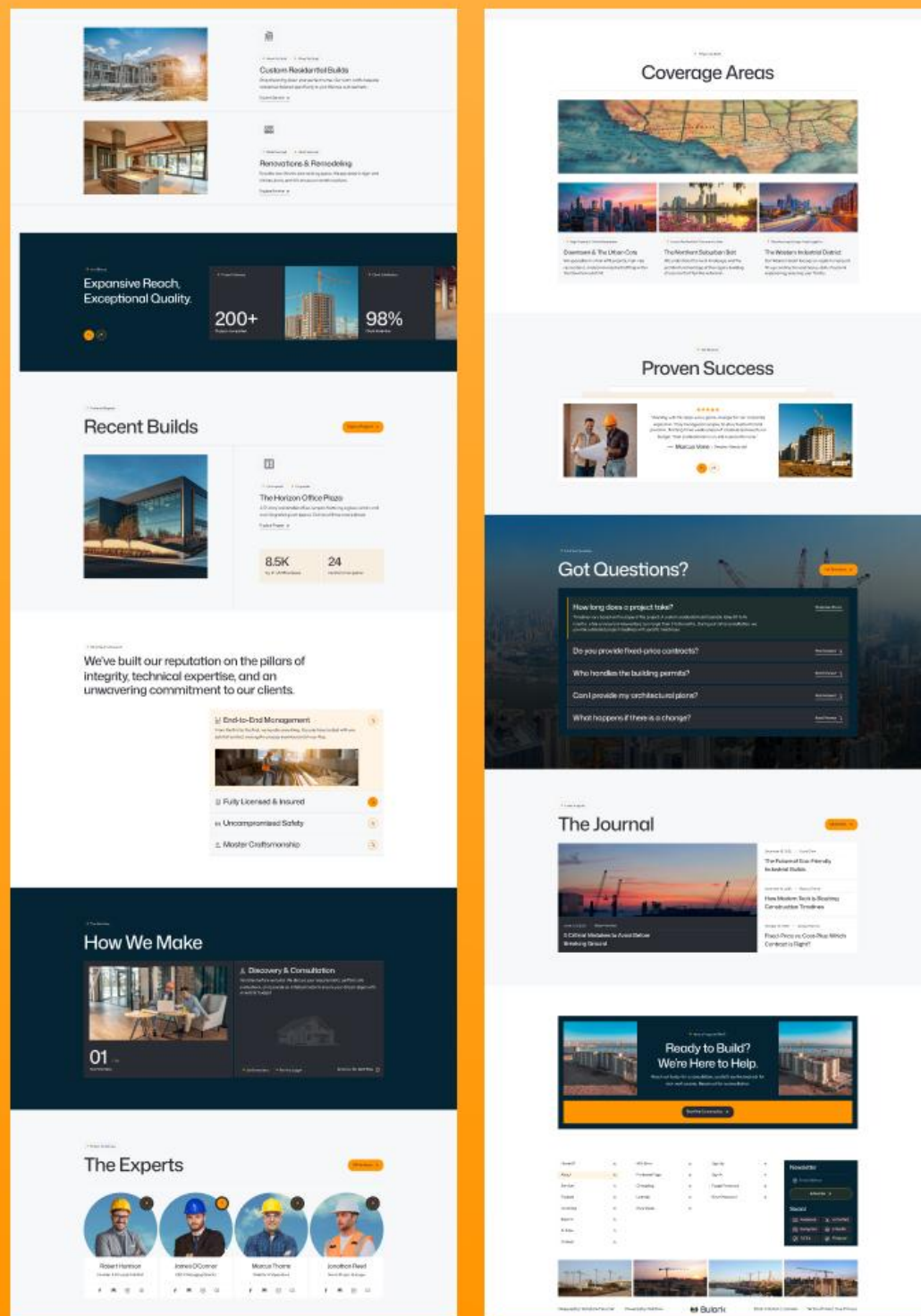
A barra de navegação desaparece quando o usuário rola a página para baixo, prejudicando a usabilidade.

Sobreposição Indevida

Elementos se sobrepõem sem controle, dificultando a leitura e interação com o conteúdo.

Sem Feedback Visual

Links e botões não mudam de aparência ao passar o mouse, deixando a interface sem interatividade.



display: Controlando o Comportamento dos Elementos

A propriedade `display` define como um elemento é renderizado no fluxo do documento. É uma das propriedades mais importantes do CSS para controle de layout.

1

`block`

Ocupa toda a largura disponível. Quebra linha antes e depois.
Ex: `div`, `p`, `h1`.

2

`inline`

Ocupa apenas o espaço do conteúdo. Não aceita `width/height`. Ex: `span`, `a`, `strong`.

3

`inline-block`

Combina o melhor dos dois: fica na mesma linha MAS aceita `width` e `height` definidos.

4

`none`

Remove o elemento completamente da tela e do fluxo. Útil para mostrar/esconder elementos com JavaScript.

Block vs. Inline: A Diferença Visual

Entender essa diferença é fundamental para controlar o layout. Veja a comparação prática aplicada ao menu do nosso e-commerce:

✗ Antes — `display: block` (padrão do `<li>`)

```
nav ul li {  
  /* comportamento padrão */  
  display: block;  
}  
  
/* Resultado: itens empilhados */  
[Início]  
[Produtos]  
[Contato]
```

Cada item de menu ocupa uma linha inteira, empurrando os próximos para baixo.

✓ Depois — `display: inline-block`

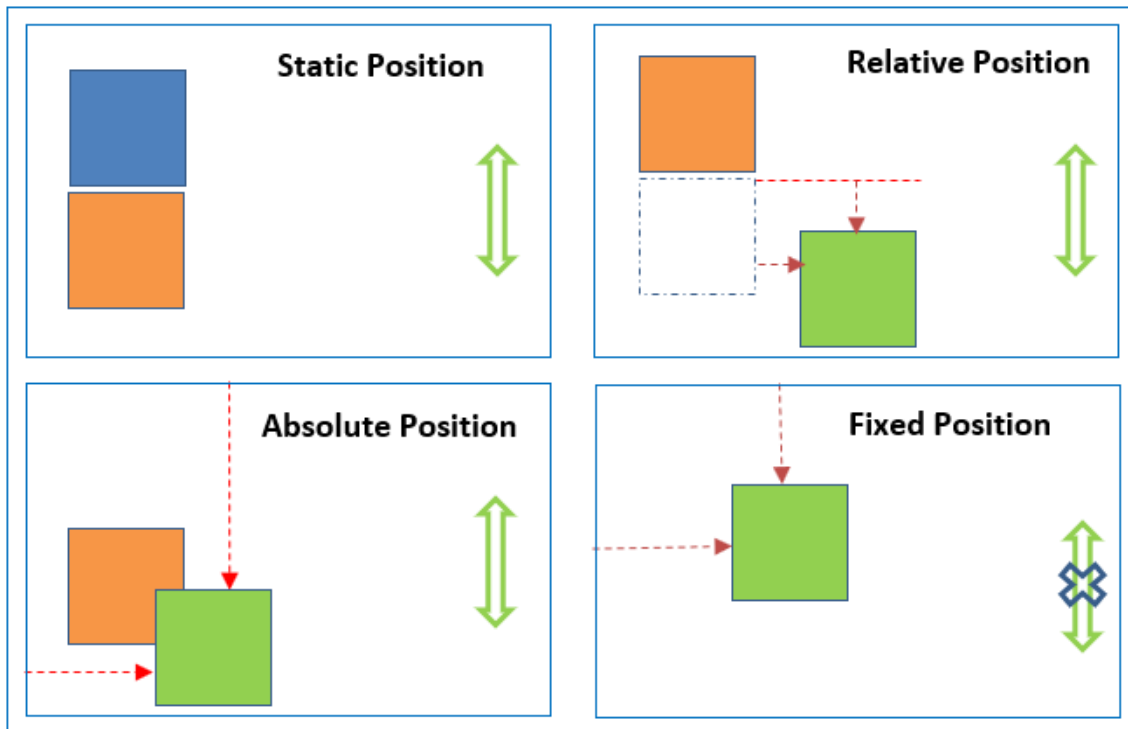
```
nav ul li {  
  display: inline-block;  
  margin-right: 16px;  
}  
  
/* Resultado: itens lado a lado */  
[Início] [Produtos] [Contato]
```

Com `inline-block`, os itens ficam alinhados horizontalmente, mas ainda aceitam padding e margin personalizados.

👉 Dica: para o menu horizontal do nosso e-commerce, usaremos `display: inline-block` nos itens da lista. Simples e eficaz!

Introdução ao Posicionamento

O CSS oferece cinco tipos de posicionamento. Cada um define como o elemento se comporta em relação ao fluxo normal da página e a outros elementos ao redor.



static

Padrão de todos os elementos. Segue o fluxo normal do documento, ignorando top/left/right/bottom.



relative

Desloca o elemento em relação à sua posição original, mantendo o espaço reservado no fluxo.



absolute

Remove do fluxo e posiciona em relação ao ancestral com position diferente de static.



fixed

Fica fixo na janela do navegador, independente de scroll. Perfeito para headers e botões flutuantes.

Relative vs. Absolute: O Par Mais Usado

A combinação `position: relative` no pai e `position: absolute` no filho é um dos padrões mais poderosos do CSS clássico. Permite posicionar elementos com precisão milimétrica.

`position: relative`

```
/* O elemento pai serve
   como referência */
.card-produto {
  position: relative;
  width: 200px;
  height: 250px;
}

/* O elemento se move
   da sua posição original */
.destaque {
  position: relative;
  top: 10px;
  left: 5px;
}
```

O espaço original do elemento é **preservado** no layout da página.

`position: absolute`

```
/* Badge "Novo!" no card */
.badge-novo {
  position: absolute;
  top: 8px;
  right: 8px;
  background: #e74c3c;
  color: white;
  padding: 4px 8px;
  border-radius: 4px;
}

/* Sai do fluxo normal!
   Outros elementos ignoram
   sua existência */
```

Ideal para **badges**, **etiquetas** e elementos decorativos nos cards de produto.

  Regra de ouro: para que o `absolute` funcione corretamente, o elemento pai DEVE ter `position: relative` definido.

position: fixed — O Header que Acompanha o Usuário

Por que usar fixed no header?

Em e-commerces reais, o menu de navegação permanece visível enquanto o usuário rola a página. Isso melhora a experiência e facilita a navegação entre categorias de produtos.

```
/* Header fixo no topo */
header {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  background: #02294d;
  z-index: 100;
  padding: 16px 24px;
  box-shadow: 0 2px 8px
    rgba(0,0,0,0.3);
}
```

⚠️ Atenção ao Scroll!

Quando o header é fixado, ele sai do fluxo normal e pode sobrepor o conteúdo. Para corrigir, adicione uma margem no topo do body:

```
/* Compensa a altura
   do header fixo */
body {
  margin-top: 72px;
  /* ajuste conforme a
     altura do seu header */
}

main {
  padding-top: 16px;
}
```

⚠️ Sempre compense a altura do header fixo no elemento subsequente para evitar que o conteúdo fique escondido!

z-index: Quem Fica Por Cima?

O `z-index` controla a ordem de empilhamento dos elementos no eixo Z (profundidade). Funciona apenas em elementos com `position` diferente de `static`.

```
/* Camadas do nosso e-commerce */

header {
  position: fixed;
  z-index: 100; /* sempre no topo */
}

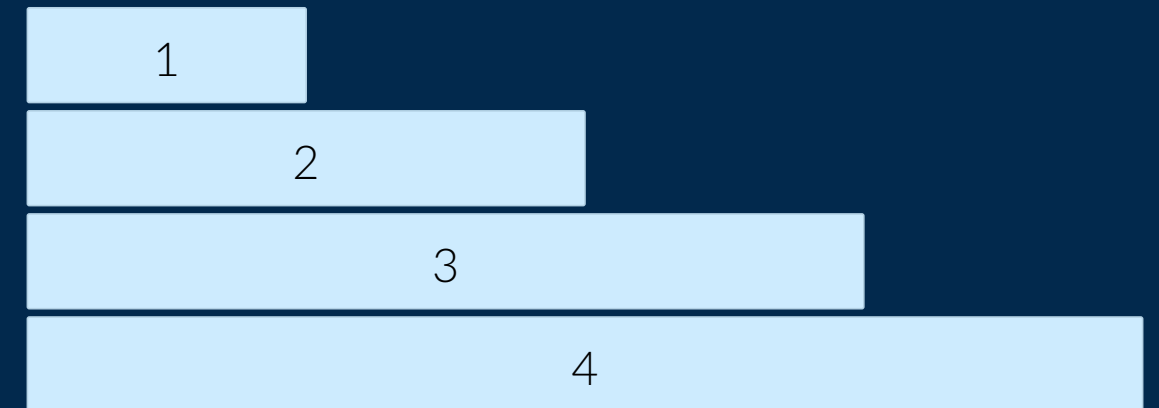
.modal-carrinho {
  position: fixed;
  z-index: 200; /* acima do header */
}

.badge-produto {
  position: absolute;
  z-index: 10; /* acima do card */
}

.card-produto {
  position: relative;
  z-index: 1; /* referência base */
}

/* Maior número = mais à frente */
```

Hierarquia Visual



- 1 z-index: 1 — Cards
- 2 z-index: 10 — Badges
- 3 z-index: 100 — Header
- 4 z-index: 200 — Modal

Pseudo-classes: :hover e :active

Pseudo-classes permitem estilizar elementos com base no estado de interação do usuário, criando feedback visual sem JavaScript.

:hover — Ao Passar o Mouse

```
/* Links da navegação */
nav a {
  color: white;
  text-decoration: none;
  padding: 8px 12px;
  transition: background 0.3s;
}

nav a:hover {
  background: #0570b5;
  border-radius: 4px;
}

/* Cards de produto */
.card-produto:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 20px
    rgba(0,0,0,0.15);
  transition: all 0.3s ease;
}
```

:active — Ao Clicar

```
/* Botão adicionar ao carrinho */
.btn-comprar {
  background: #0570b5;
  color: white;
  border: none;
  padding: 10px 20px;
  cursor: pointer;
  transition: all 0.1s;
}

.btn-comprar:hover {
  background: #02294d;
}

.btn-comprar:active {
  transform: scale(0.97);
  background: #51c8e6;
  /* simula "clique físico" */
}
```

✔ A propriedade `transition` suaviza as mudanças entre estados, tornando a interface muito mais profissional e agradável.

Antes e Depois: O E-Commerce Evoluindo

Antes

Layout desorganizado, menu vertical, header que some



Depois

Menu horizontal, header fixo, hover e badges posicionadas



A aplicação das técnicas de hoje transforma significativamente a aparência e usabilidade do e-commerce. Cada propriedade aprendida resolve um problema visual concreto do projeto.

Sua Missão: Evoluir o E-Commerce

1 Menu Horizontal

Transforme a lista de navegação em um menu horizontal usando `display: inline-block` nos itens `<li>`.

2 Header Fixo

Aplique `position: fixed` no `<header>` e ajuste o `margin-top` do conteúdo para compensar.

3 Efeito Hover

Crie efeitos visuais nos links de navegação e nos cards de produto usando `:hover` e `transition`.

4 Badge de Produto

Adicione uma etiqueta "Novo!" posicionada com `position: absolute` no canto superior de um card.

Erros Comuns

Armadilhas do Posicionamento CSS

Conhecer os erros mais comuns evita horas de depuração. Fique atento a estes casos ao desenvolver seu e-commerce:

✗ Absolute sem Relative no Pai

O elemento se posiciona em relação ao `<body>` inteiro, não ao card. Solução: adicione `position: relative` no elemento pai.

✗ Fixed sem Compensar o Scroll

O header fixo cobre o início do conteúdo. Solução: adicione `margin-top` ou `padding-top` no `<body>` com o valor igual à altura do header.

✗ z-index sem Position Definido

O `z-index` é ignorado em elementos com `position: static`. Solução: sempre defina `position: relative/absolute/fixed` antes de usar `z-index`.

✗ transition sem Declarar no Estado Base

A animação só funciona em uma direção. Solução: sempre declare a propriedade `transition` no seletor base do elemento, não no `:hover`.

O Que Vem a Seguir

Próxima Aula: Flexbox

→ display: flex no container

Como transformar um elemento em um container flexível e controlar seus filhos com propriedades como `justify-content` e `align-items`.

→ flex-direction e flex-wrap

Controlar a direção do fluxo (linha ou coluna) e o comportamento quando os itens não cabem em uma única linha.

→ Layouts responsivos com Flexbox

Criar grids de produtos que se adaptam automaticamente ao tamanho da tela, eliminando a necessidade de larguras fixas.

Resumo da Aula 7

Você aprendeu as ferramentas fundamentais do posicionamento CSS e já sabe aplicá-las no projeto real de e-commerce. Veja o que foi coberto:



Propriedade display

`block`, `inline`, `inline-block` e `none` — controlam como o elemento ocupa espaço no fluxo da página.



z-index e Camadas

Controle a ordem de sobreposição dos elementos para criar hierarquias visuais corretas e consistentes.



Tipos de Posicionamento

`static`, `relative`, `absolute` e `fixed` — cada um com comportamento específico e casos de uso distintos.



Pseudo-classes

`:hover` e `:active` com `transition` — adicionam interatividade e feedback visual sem JavaScript.

